

Section 1

For this assignment, we designed the test suite to check the game model at three levels: normal functionality, edge cases, and invariants. We first tested expected behaviors such as valid movement, invalid move rejection, and correct turn changes. We then focused on situations that were more likely to expose hidden errors, including repeated extra turns, duplicate throw values, boundary stations, captures, stacking, and consecutive red-X moves. These cases were important because AI-generated code can appear correct during normal play while still failing under unusual state combinations.

Our knowledge of the HW2 generation process influenced the test design because the implementation was produced from prompts and then extended with a Flutter interface. This meant we could not assume that all rules were implemented consistently or that the model and UI used the same assumptions. We therefore tested the model classes directly instead of relying on visible UI behavior. We also compared the generated behavior against the HW1A contract and invariants, especially the requirements for 29 stations, four pieces per player, valid piece positions, and valid available throws. The duplicate-throw test was designed specifically to detect a non-obvious failure that would not be noticeable during simple gameplay.

Section 2

The test suite revealed four failures in the generated game model. The first and most important failure involved duplicate throw values. The test named NON-OBVIOUS FAILURE: using one of two equal throws preserves the other set availableThrows to [2, 2], moved one piece using a value of 2, and then asserted that one 2 should remain. The key assertion expected [2], but the actual result was an empty list. This means that one move consumed both identical throws instead of exactly one. I classified this as a hallucination because the generated code introduced behavior that was not required by the rules and directly contradicted the turn-state contract. The root cause was an AI-generated duplicate removal operation in movePiece(), which removed the same move value twice.

The second failure involved the extra-turn rule. The test a result of 4 grants an extra turn after its move is used expected the current player to remain Player 1 after using a throw value of 4. Instead, the actual current player became Player 2. I classified this as a hallucination because the rule explicitly states that both 0 and 4 grant an extra turn, but the generated implementation only handled the case that maps to a move value of 5. The likely root cause was an AI assumption that only one special throw result needed to be implemented, even though the specification required two.

The third failure occurred in the physical stick model. The test throwSticks models the four physical sticks in the rules asserted that lastSticks should have a length of 4. The actual result contained 5 Boolean values. I classified this as a hallucination because the implementation invented an additional stick that was not part of the specified game. The root cause was not an ambiguous rule; the requirement was clear, but the model generated a common-looking random list without preserving the exact physical constraint.

The fourth failure involved consecutive red-X backward movement. The test consecutive red-X moves continue backward along the previously used path expected two backward moves to complete normally. Instead, the second move threw an `ArgumentError` stating that station 1 did not connect to station 1. I classified this as underspecification because the prompt described backward movement but did not clearly state how `previousStation` should be updated after each backward step. The implementation changed the piece's current position but left its movement history stale. As a result, the next backward move used inconsistent state and attempted an invalid self-connection. This failure suggests that the prompt needed a more explicit postcondition for backward movement.

Section 3

Classification: Hallucination

Root cause: The generated `movePiece()` method duplicated the removal operation. The rules require one move to consume exactly one available throw, but the model introduced an extra state change that was never specified.

Corrected prompt: IMPORTANT: EACH call to `movePiece(piece, moveValue)` MUST remove EXACTLY ONE occurrence of `moveValue` from `availableThrows`. If `availableThrows` is `[2, 2]`, one move using 2 MUST leave `[2]`. DO NOT remove the selected value more than once. Include a unit test for duplicate throw values.

Failure: A throw result of 4 changes the turn instead of granting the current player an extra turn.

Classification: Hallucination

Root cause: The generated implementation handled the special case for zero marked sticks but omitted the separate rule for four marked sticks, even though both outcomes grant an extra turn.

Corrected prompt: EXTRA-TURN RULE: A result of EITHER 0 marked sticks OR 4 marked sticks MUST grant an extra turn. After the player uses the corresponding move, currentPlayer MUST remain unchanged. Implement and test both outcomes independently.

Failure: The engine generates five physical sticks instead of four.

Classification: Hallucination

Root cause: The model generated a list of five Boolean values even though the game specification defines exactly four physical sticks. This was an unsupported implementation choice that contradicted the stated rules.

Corrected prompt: THE GAME USES EXACTLY FOUR PHYSICAL STICKS. lastSticks MUST always contain exactly four Boolean values after throwSticks() is called. Compute the move value only from those four values. NEVER generate a fifth stick. Add a test asserting lastSticks.length == 4.

Failure: A second consecutive red-X backward move throws an exception because the piece's movement history is stale.

Classification: Underspecification

Root cause: The prompt described backward movement but did not clearly define the required postcondition for previousStation. The generated code updated the piece's current position without updating the history needed for another backward move.

Corrected prompt: AFTER EVERY BACKWARD MOVE, update BOTH position and previousStation so the piece's path history remains consistent. For a path 0 -> 1 -> 2, two consecutive backward moves MUST produce 2 -> 1 -> 0 without an exception. Add a unit test that performs two backward moves in sequence.

Section 4

One LLM-assisted test was misleading because it reused the same piece during five consecutive extra-turn moves. The test was intended to check whether the current player remained unchanged after repeated extra-turn results. However, after several moves, the selected piece reached the completed state. The next move then correctly threw the exception Cannot move a completed piece, which initially appeared to be a failure in the extra-turn logic.

We detected the problem by reading the exception message and tracing the piece's position through each loop iteration. This showed that the failure came from the test setup rather than the game behavior being tested. We changed the test so that the moves rotated among the player's four pieces instead of repeatedly moving one piece. This prevented any single piece from completing and allowed the test to focus only on turn preservation. After the change, the test passed, confirming that the original result was a false alarm caused by an incorrect LLM-generated test assumption.